UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026

INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Tema de la guía práctica proporcionada por el docente
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	1 - B
Alumnos participantes:	Alulema Chato Sandra Jazmin
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. Jose Ruben Caiza Caizabuano

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:
Desarrollar programas en Java que permitan crear, escribir y eliminar archivos de texto, aplicando las clases y métodos básicos para el manejo de archivos.
Específicos:

- Utilizar File y createNewFile() para crear un archivo y comprobar su existencia.
- Leer información por teclado y almacenarla mediante FileWriter conservando el contenido anterior.
- Aplicar delete() y estructuras condicionales para comprobar la eliminación.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6
No presenciales: 0

2.4 Instrucciones

Desarrolle los 3 ejercicios indicadados en clase de manera individual. Realice un archivo en pdf y coloque el link del repositorio del github.

APE 5 primera parte.docx

2.5 Listado de equipos, materiales y recursos

- Computadora
- JDK y entorno de desarrollo Java.
- Editor o IDE de programación.
- GitHub para alojar el código fuente.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☐Simuladores y laboratorios virtuales
☐Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☐Inteligencia Artificial
Otros (Especifique): _____

2.6 Actividades por desarrollar

Ejercicio 1. Crear un fichero en Java

Enunciado:

Desarrollar un programa en Java que permita crear un archivo de texto llamado curso.txt.

Indicaciones:

- Importar las clases necesarias para trabajar con archivos.
- Crear un objeto de tipo File.
- Utilizar el método createNewFile() para crear el archivo.
- Comprobar si el archivo se creó correctamente o si ya existe.
- Mostrar el nombre del archivo utilizando getName().
- Utilizar try-catch para controlar posibles errores.

Clases y métodos sugeridos:

- File
- IOException
- createNewFile()
- getName()

Código Java:

```
import java.io.File;
import java.io.IOException;

public class CrearArchivo {
    public static void main(String[] args) {
        // Se agregaron las comillas a "curso.txt"
        File archivo = new File("curso.txt");

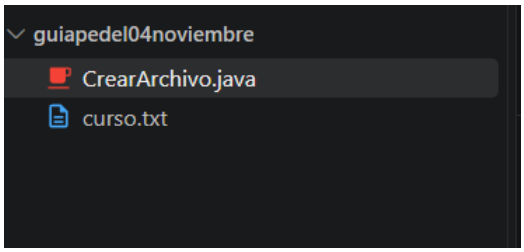
        try {
            // Intenta crear el archivo en el disco
            if (archivo.createNewFile()) {
                System.out.println("Archivo creado correctamente.");
            } else {
                System.out.println("El archivo ya existe.");
            }
            System.out.println("Nombre del archivo: " +
archivo.getName());

        } catch (IOException e) {
            System.out.println("Ocurrió un error al crear el archivo.");
            e.printStackTrace();
        }
```

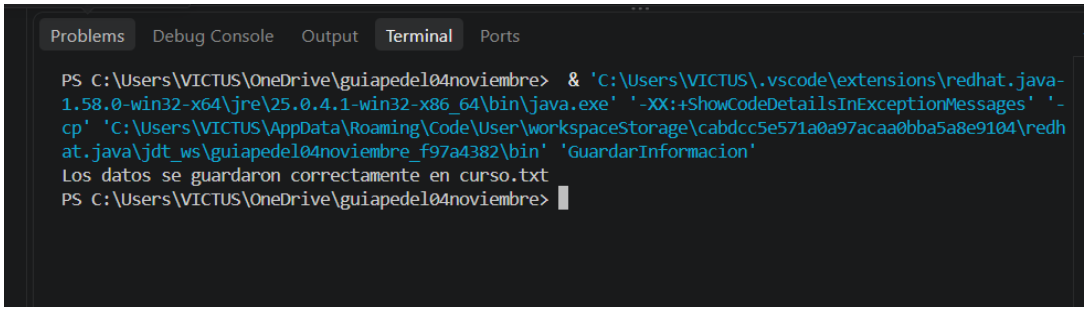
```
    }
}
```

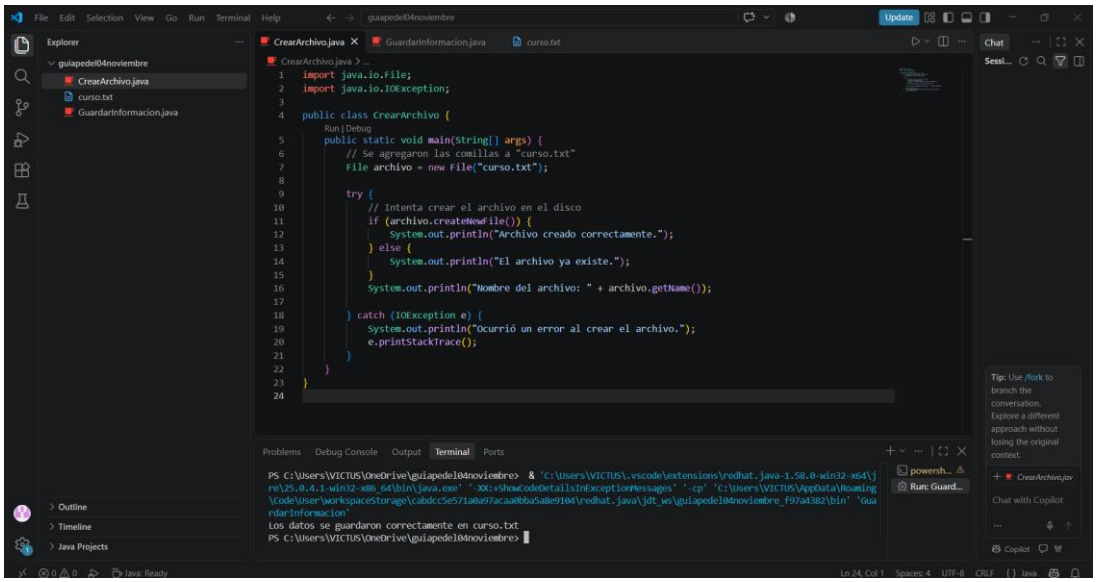
Resultado esperado:

Curso.txt



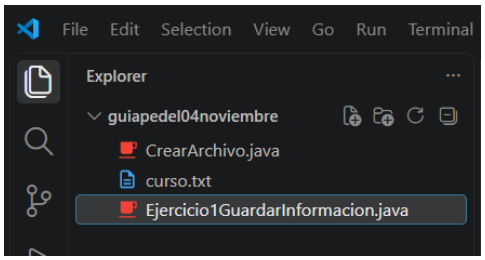
Validar:

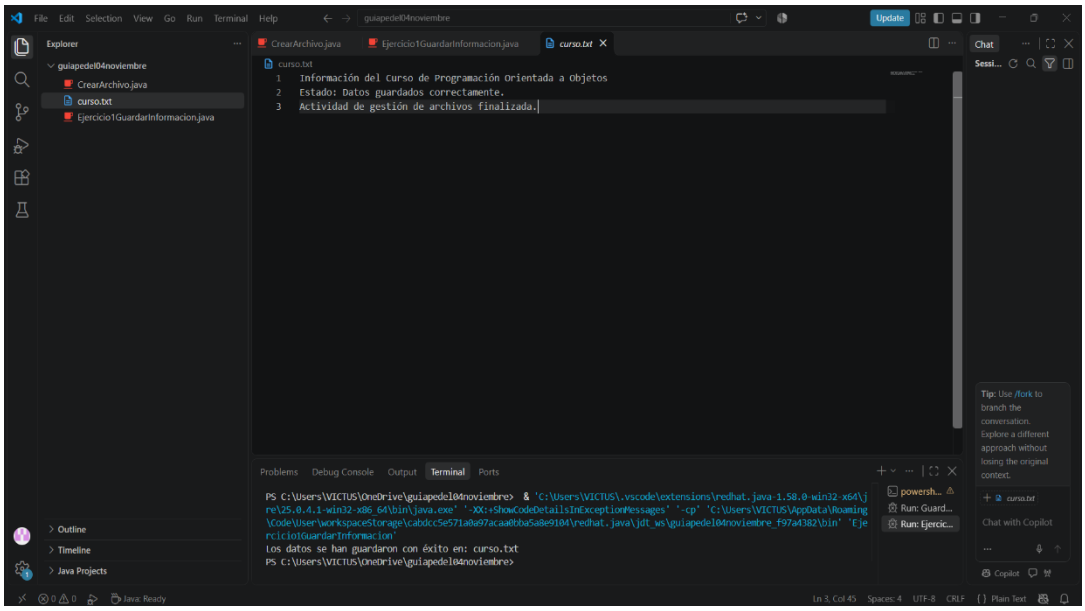




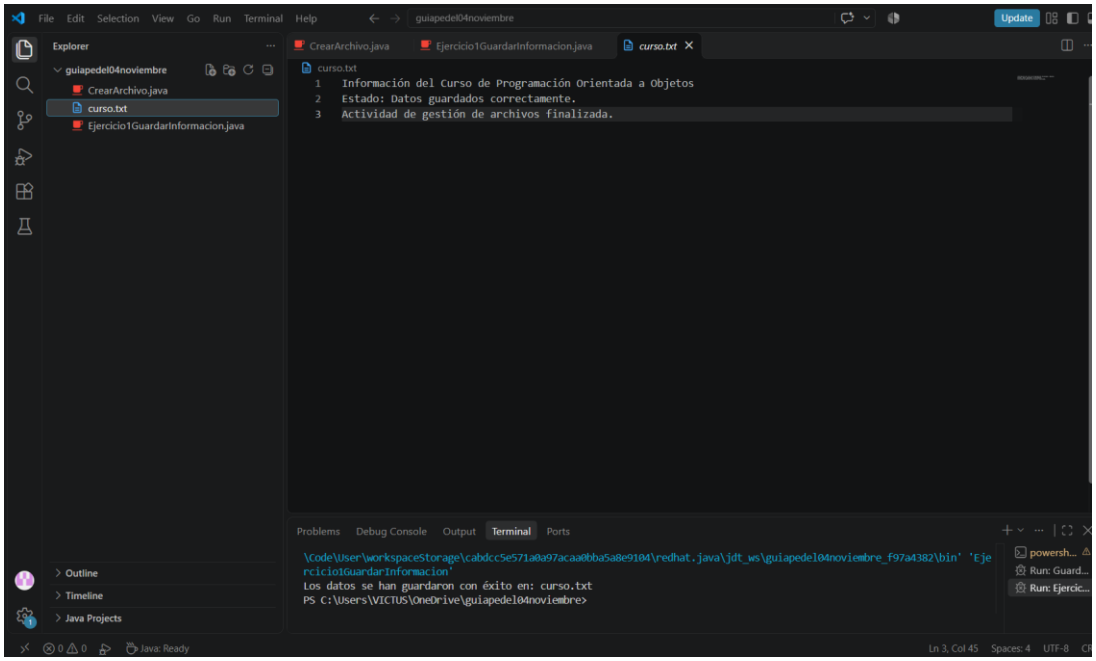
Y si guardamos información:

Ejercicio1GuardarInformacion.java





La información se genera en curso.txt



Ejercicio 2. Escribir información en un fichero

Enunciado:

Desarrollar un programa en Java que permita ingresar un mensaje por teclado y almacenarlo en un archivo de texto existente llamado estudiantes.txt.

Indicaciones:

- Importar las clases necesarias.
- Crear un objeto Scanner para ingresar información por teclado.
- Solicitar al usuario que escriba un mensaje.
- Utilizar FileWriter para abrir el archivo estudiantes.txt.
- Utilizar el parámetro true para conservar la información anterior y agregar el nuevo mensaje al final.
- Utilizar el método write() para guardar el mensaje.
- Cerrar el archivo utilizando close().
- Mostrar un mensaje indicando que la información se guardó correctamente.
- Controlar posibles errores mediante try-catch.

Clases y métodos sugeridos:

- Scanner
- FileWriter
- IOException
- write()
- close()

Nota: El archivo estudiantes.txt debe existir previamente.

Código Java:

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class EscribirArchivo {
    public static void main(String[] args) {
        // 1. Crear un objeto Scanner para ingresar información por
teclado
        Scanner teclado = new Scanner(System.in);

        // 2. Solicitar al usuario que escriba un mensaje
        System.out.print("Ingrese un mensaje: ");
        String mensaje = teclado.nextLine();

        // 3. Controlar posibles errores mediante try-catch
        try {
```

```
            // 4. Utilizar FileWriter para abrir el archivo
estudiantes.txt
            // 5. Utilizar el parámetro true para conservar la
información anterior (modo append)
            FileWriter escritor = new FileWriter("estudiantes.txt",
true);

            // 6. Utilizar el método write() para guardar el mensaje
junto con un salto de línea
            escritor.write(mensaje + System.lineSeparator());

            // 7. Cerrar el archivo utilizando close()
            escritor.close();

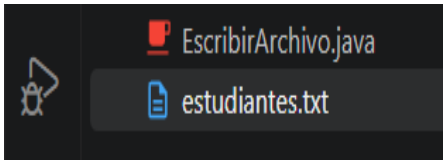
            // 8. Mostrar un mensaje indicando que la información se
guardó correctamente
            System.out.println("La información se guardó
correctamente.");

        } catch (IOException e) {
            System.out.println("No se pudo escribir en el archivo.");
            e.printStackTrace();
        }

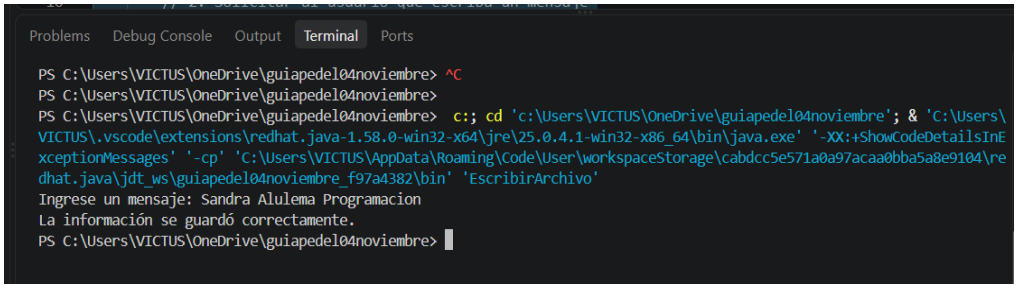
        // Cerrar el objeto Scanner
        teclado.close();
    }
}
```

Resultado esperado:

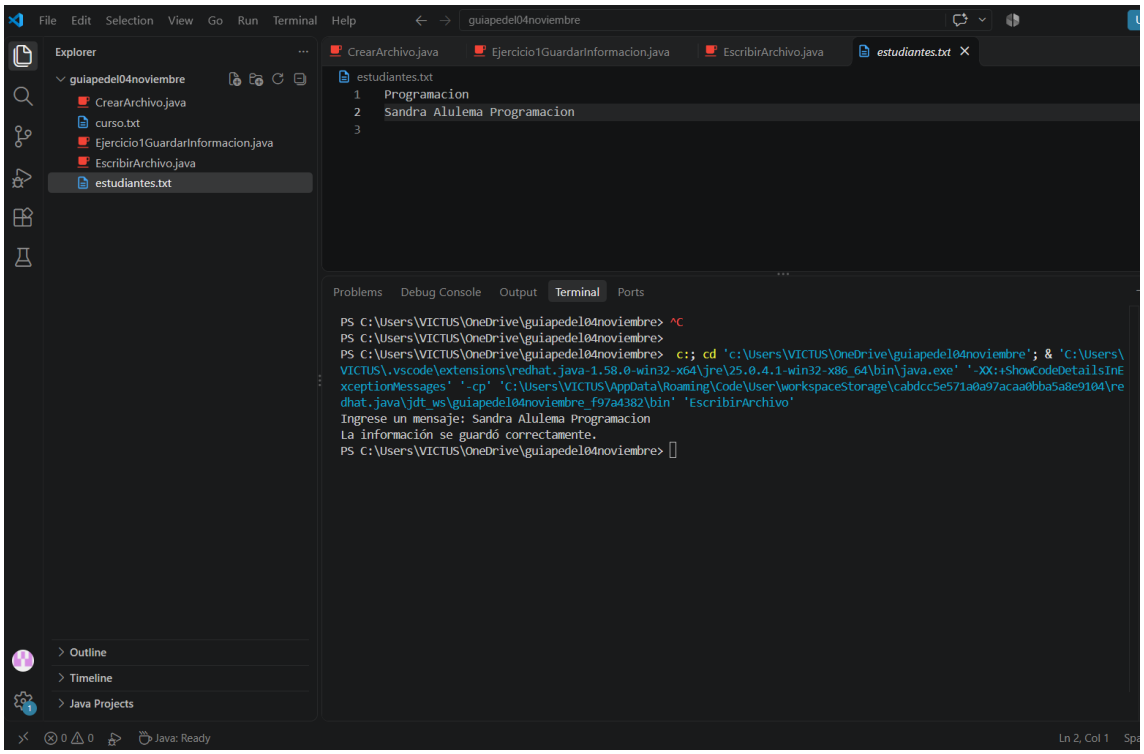
estudiantes.txt



Validar y Ejecutar:



La información se genera en estudiantes.txt



Ejercicio 3. Eliminar un fichero en Java

Enunciado:

Desarrollar un programa en Java que permita eliminar el archivo de texto estudiantes.txt.

Indicaciones:

- Importar la clase necesaria para trabajar con archivos.
- Crear un objeto de tipo File indicando el nombre del archivo que se desea eliminar.
- Utilizar el método delete() para intentar eliminar el archivo.
- Comprobar si la eliminación se realizó correctamente.
- Si el archivo se eliminó, mostrar un mensaje de confirmación.
- Si no se pudo eliminar, mostrar un mensaje indicando el problema.

Clases y métodos sugeridos:

- File
- delete()
- if-else
- System.out.println()

Código Java:

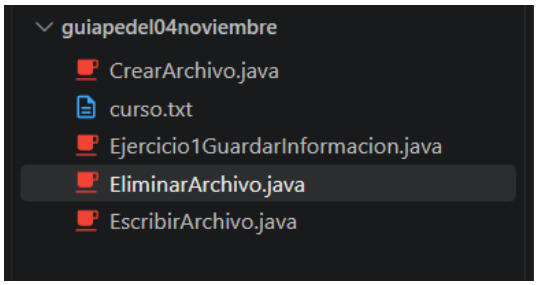
```
import java.io.File;

public class EliminarArchivo {
    public static void main(String[] args) {

        // 1. Crear un objeto de tipo File indicando el nombre del
archivo a eliminar
        File archivo = new File("estudiantes.txt");

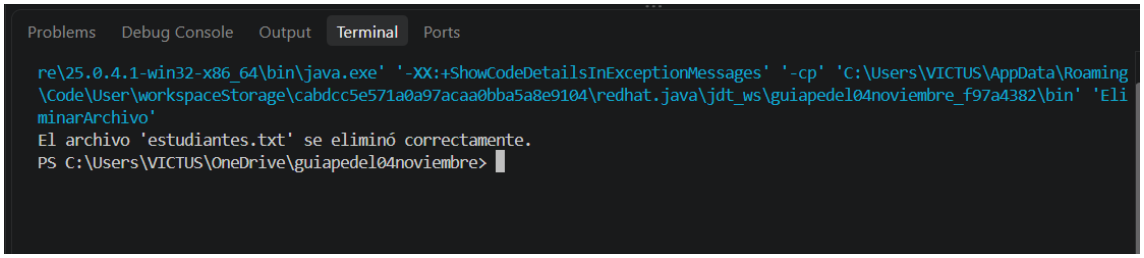
        // 2. Utilizar el método delete() junto con un bloque if-else
para comprobar el resultado
        if (archivo.delete()) {
            // 3. Si el archivo se eliminó, mostrar un mensaje de
confirmación
            System.out.println("El archivo 'estudiantes.txt' se eliminó
correctamente.");
        } else {
            // 4. Si no se pudo eliminar, mostrar un mensaje indicando el
problema
            System.out.println("No se pudo eliminar el archivo. Verifique
si el archivo existe o si está siendo usado por otro programa.");
        }
    }
}
```

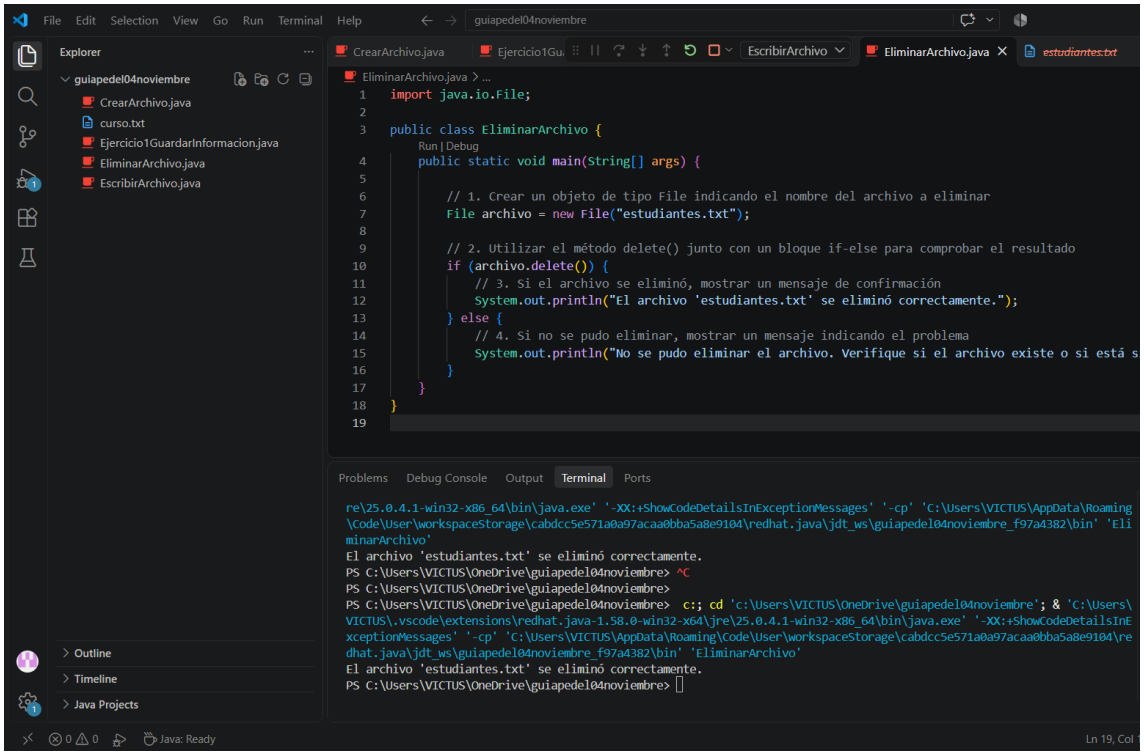
Resultado esperado:



estudiantes.txt eliminarlo

Validar:





2.7 Resultados obtenidos

La práctica permite reconocer el proceso básico de manipulación de archivos desde Java. En el primer ejercicio se crea curso.txt y se comprueba si ya existía. En el segundo se recibe un mensaje por teclado y se añade al final de estudiantes.txt mediante FileWriter en modo append (true). En el tercero se intenta eliminar

estudiantes.txt y se informa si la operación tuvo éxito. Los mensajes indicados son resultados esperados; deben complementarse con capturas de las ejecuciones realizadas.

2.8 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☐ Trabajo en equipo
☐ Comunicación asertiva
☐ La empatía
☐ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☒ Adaptabilidad
☒ Responsabilidad

2.9 Conclusiones

1. Se aplicaron las clases y métodos de Java para crear, consultar el nombre, escribir y eliminar archivos de texto.
2. try-catch permite controlar excepciones de entrada/salida y presentar mensajes al usuario.
3. El parámetro true de FileWriter permite añadir mensajes al final sin reemplazar la información existente.

2.10 Recomendaciones

- Verificar que estudiantes.txt exista antes de ejecutar el segundo ejercicio.
- Ejecutar los programas en orden y comprobar los archivos en la carpeta de trabajo. • Agregar capturas de pantalla de cada ejecución como evidencia.
- Antes del tercer ejercicio, confirmar que se desea eliminar estudiantes.txt.

2.11 Referencias bibliográficas

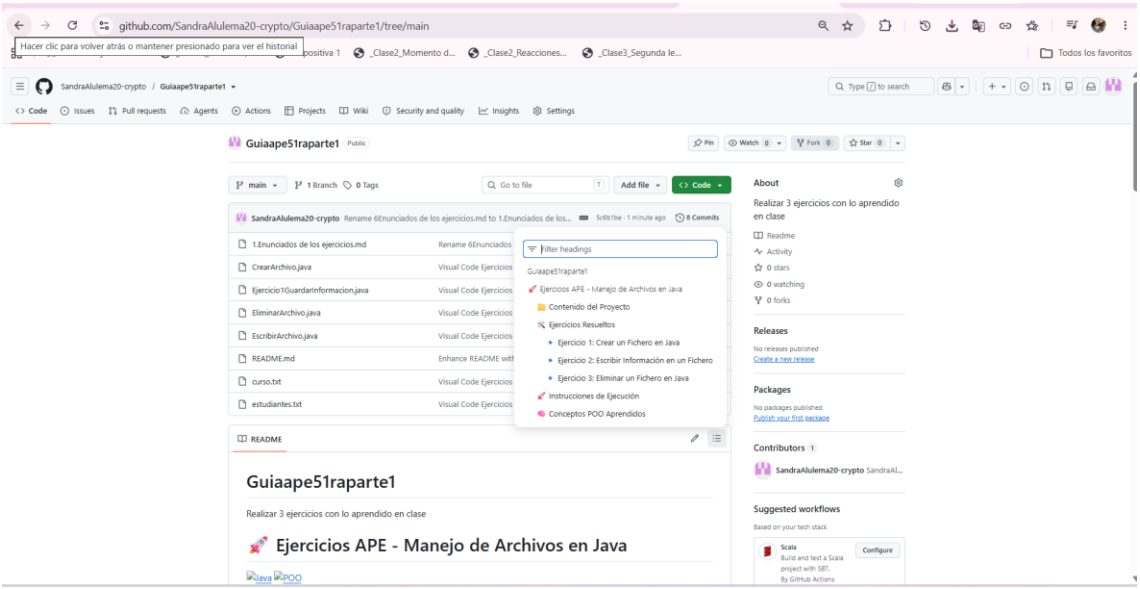
2.11 Referencias bibliográficas – Formato IEEE

[1] Universidad Técnica de Ambato, Facultad de Ingeniería en Sistemas, Electrónica e Industrial, Guía Práctica APE 5: Manejo de archivos en Java, Carrera de Software, ciclo académico julio–diciembre de 2026.

[2] Universidad Técnica de Ambato, APE 5 – Primera parte, Sistema Integrado de Educación en Línea, 2026. En línea. Disponible en: https://sistemaseducaciononline.uta.edu.ec/mod/assign/view.php?id=34912

[3] Universidad Técnica de Ambato, Algoritmos y Lógica de Programación, material de apoyo académico, Carrera de Software, 2026.

2.12 Anexos



https://github.com/SandraAlulema20-crypto/Guiaape51raparte1/tree/main